컴퓨터 네트워크 Assignment 1 디자인 및 구현

학번: 2022063672
이름: 박수현

1 전체 source directory 구조와 주요 class 설명

1.1 source 및 제출 directory 구조

```
Assignment1_2022063672_박수현/
├── src/
│   ├── MyWebServer.java
│   ├── http/
│   │   ├── HttpParser.java
│   │   └── HttpRequest.java
│   └── webserver/
│       ├── ClientHandler.java
│       ├── GetHandler.java
│       ├── PutHandler.java
│       └── PostHandler.java
├── docs/
│   ├── public/
│   └── upload/
└── 디자인및구현문서.pdf
```

1.2 주요 class

MyWebServer: ServerSocket 으로 연결을 수락하고 10 개의 작업 스레드의 풀에 ClientHandler 실행을 맡긴다.

HttpParser: 요청 줄, 헤더와 Content-Length 길이의 본문을 읽어 HttpRequest 를 생성한다.

HttpRequest: 메서드, URI, HTTP 버전, 헤더와 본문을 객체 형태로 보관한다.

ClientHandler: Runnable 로 요청을 처리하고 메서드별 핸들러 호출, 업로드 동기화와 응답 전송을 담당한다.

GetHandler: docs/public 에서 요청 파일을 읽어 응답 본문으로 전송한다.

PutHandler: URI 의 파일명으로 요청 본문 전체를 docs/upload 에 저장한다.

PostHandler: multipart 의 file 파트에서 filename 과 파일 바이트를 추출해 docs/upload 에 저장한다.

2 HTTP request parsing 과 response 메시지 작성 방법

2.1 HTTP request parsing

HttpParser.readRequest 는 요청 줄을 method, URI, HTTP version 으로 나눈다. readLine 은 CRLF 단위로 읽고, readHeaders 는 빈 줄까지 헤더를 읽어 대소문자를 구분하지 않는 TreeMap 에 저장한다.

readBody 는 Content-Length 에 지정된 바이트 수를 모두 받을 때까지 반복해서 읽는다. 본문을 byte 배열로 유지하여 바이너리 파일 손상을 막고, Content-Length 가 없으면 길이를 0 으로 처리한다. 형식 오류, 불완전한 수신, 중복 Content-Length 와 Transfer-Encoding 은 400 Bad Request 로 처리한다.

2.2 HTTP response 메시지 작성

ClientHandler.sendResponse 는 HTTP/1.1 상태 줄, 헤더, 빈 줄, 본문 순서로 출력한다. 본문은 byte 배열 그대로 쓴다. Content-Length 는 본문 바이트 수이며 Connection: close 를 포함한다.

GET 의 Content-Type 은 Files.probeContentType 으로 확인하고, 알 수 없으면 application/octet-stream 을 사용한다. sendStatus 는 본문 없는 응답을 만든다. 전송 후 flush 하고 소켓을 닫는다.

성공은 200 또는 201, 잘못된 요청은 400, 없는 파일은 404, 미지원 메서드는 405 로 응답한다. 405 에는 Allow: GET, PUT, POST 를 추가한다.

3 Thread-safe 설계와 쓰레드 동기화 및 synchronized access 기법

3.1 Thread-safe 프로그램 설계

MyWebServer 는 Executors.newFixedThreadPool(10)으로 작업 스레드를 재사용한다. accept 로 얻은 소켓마다 ClientHandler 를 생성하여 풀에 전달하며, 소켓과 요청 데이터는 작업별로 관리해 다른 요청의 상태를 직접 수정하지 않는다.

3.2 쓰레드 동기화와 synchronized access

파일 쓰기에 synchronized (FILE_WRITE_LOCK)을 적용한다. 한 스레드가 모니터 잠금을 획득하면 다른 스레드는 대기하며, 블록 종료 또는 예외 발생 시 잠금이 자동 해제된다.

FILE_WRITE_LOCK 은 static final Object 이므로 모든 ClientHandler 가 공유한다. PUT 과 POST 의 저장을 writeUpload 로 모아 서로의 쓰기가 겹치지 않게 하고, 요청 파싱과 응답 전송은 잠금 밖에서 수행한다.

4 GET/PUT/POST URI 와 file upload/download 처리 방법

4.1 GET URI 와 file download

GET /index.html 과 GET /filename=index.html 은 ./docs/public/index.html 을 읽는다. GetHandler 는 쿼리와 filename= 접두사를 제거해 경로를 만든다. Files.readAllBytes 로 읽은 파일을 200 OK 본문으로 보내며, 대상이 일반 파일이 아니면 404 Not Found 를 반환한다.

4.2 PUT URI 와 file upload

PUT /upload/sample.bin 은 ./docs/upload/sample.bin 에 저장한다. PutHandler 는 URI 의 마지막 / 뒤를 파일명으로 사용하고 본문 전체를 writeUpload 에 전달한다. 잠금 안에서 확인한 기존 파일 여부에 따라 신규 생성은 201 Created, 덮어쓰기는 200 OK 로 응답한다.

4.3 POST URI 와 multipart file upload

POST /upload 는 multipart/form-data 의 name="file"인 파일 파트 하나를 처리한다. PostHandler 는 Content-Type 의 boundary 로 구간을 나누고, Content-Disposition 의 filename 에서 저장 파일명을 얻는다.

ISO-8859-1 문자열로 boundary 위치를 찾은 뒤 원본 byte 배열에서 헤더와 구분자를 제외한 파일 데이터만 추출한다. filename 의 마지막 경로 요소를 사용해 writeUpload 로 저장하고 201 Created 로 응답한다. file 파트 누락이나 잘못된 형식은 400 Bad Request 로 처리한다.

업로드 파일은 docs/upload 에 저장되고 GET 은 docs/public 에서 파일을 읽는다. 따라서 업로드 결과는 docs/upload 의 저장 파일로 확인한다.

5 공유 data structure 와 공유 업로드 디렉토리의 동기화

5.1 공유 data structure

공유 객체는 FILE_WRITE_LOCK 이고, 공유 자원은 docs/upload 디렉토리와 파일이다. 헤더 맵과 본문은 요청별 HttpRequest 에 보관한다. 헤더는 복사한 읽기 전용 맵으로 만들고 본문은 생성자와 getBody 에서 복사해 외부 변경을 막는다.

5.2 공유 디렉토리의 file upload synchronization

writeUpload 는 잠금 안에서 디렉토리 생성, 파일 존재 확인, 선택적 지연, Files.write 를 수행한다. 존재 확인과 저장이 한 임계 구역이므로 신규 생성 여부 판단 도중 다른 쓰기가 끼어들지 않는다.

디렉토리 전체에 하나의 잠금을 적용하여 PUT·POST 쓰기를 순차 처리한다. 따라서 요청 간 파일 데이터가 섞이지 않으며, 같은 이름의 파일은 마지막 쓰기의 내용으로 덮어쓴다.

--slowDown 500 은 잠금을 얻은 뒤 쓰기 전에 500ms 대기한다. 작업 스레드 이름과 파일명의 ENTER/EXIT 로그로 순차 진입을 확인하며, 옵션을 생략하면 인위적인 지연은 없다.

6 단계 #1 기능 테스트 결과 (캡처 포함)

6.1 테스트 환경과 실행

운영체제: Windows 10 Education 22H2 / IDE: IntelliJ IDEA 2026.2.4 / Java: Oracle JDK 25.0.4 / Postman: 12.36.0

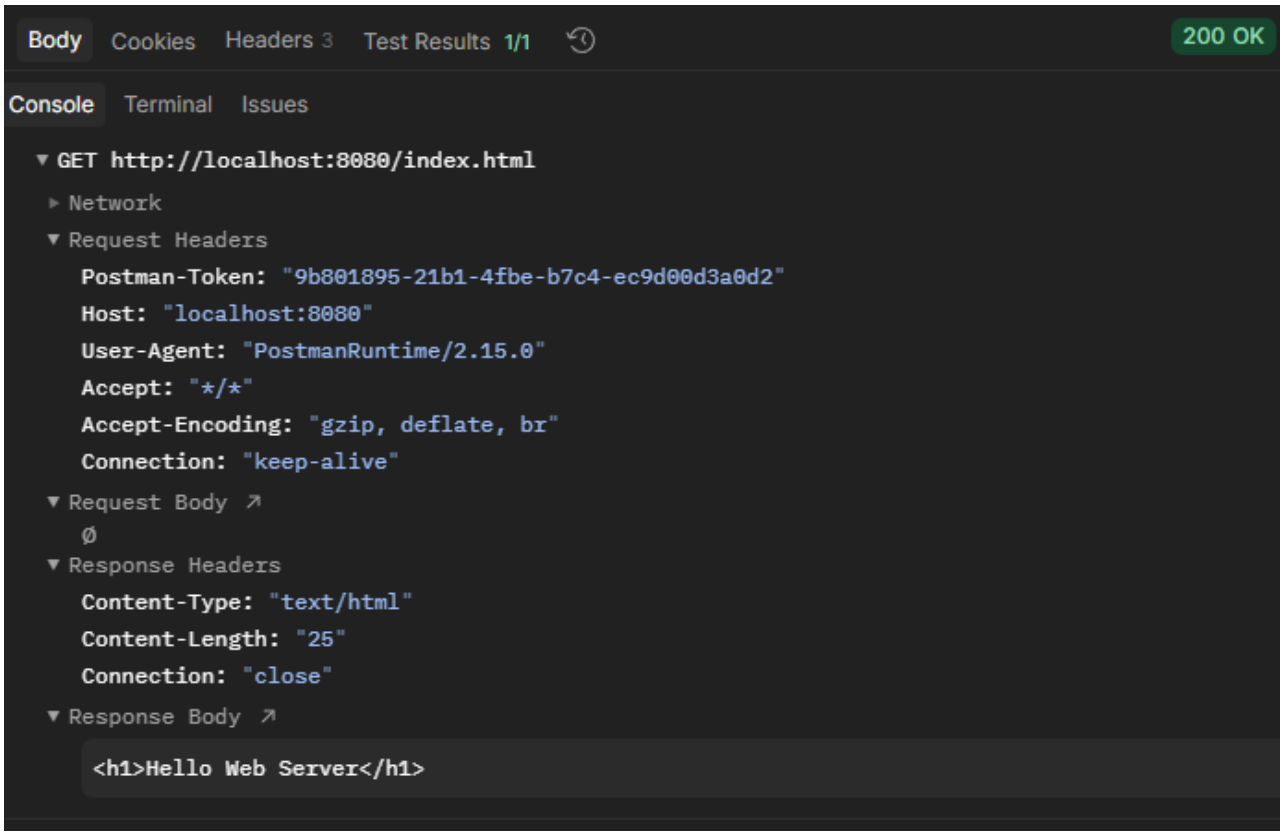
Assignment1_2022063672_박수현 폴더를 작업 디렉토리로 연 상태에서 아래 명령으로 컴파일하고 서버를 실행한다. 기능 테스트는 Postman 으로 수행하고, Postman Console 에서 실제 method, URL, header 와 body 도 확인한다.

6.2 실행방법:

```
cd .\Assignment1_2022063672_박수현\
javac -d out -sourcepath src src/MyWebServer.java
java -cp out MyWebServer --port 8080
```

6.3 WS-01 GET 파일 다운로드

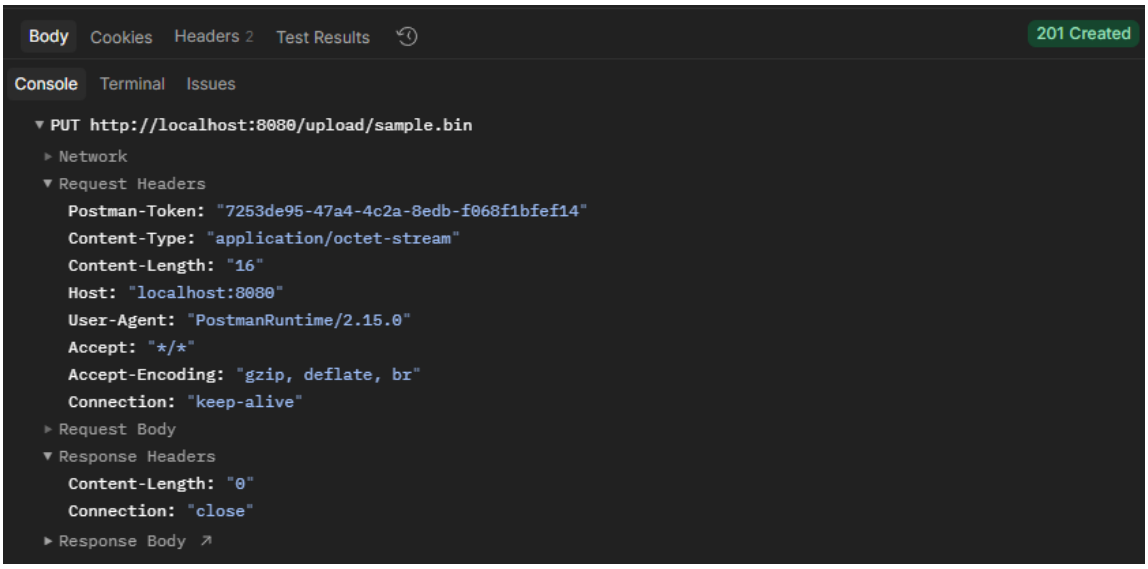
GET http://localhost:8080/index.html 을 전송한다. 200 OK, Content-Type, 본문 바이트 수와 일치하는 Content-Length, HTML 본문을 확인한다.



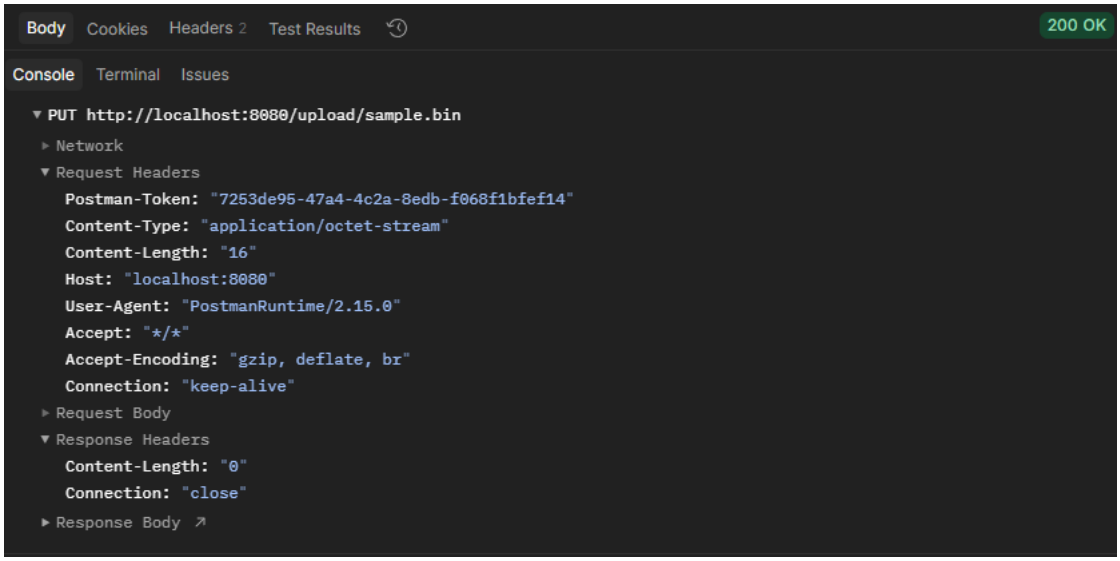
6.2.1 GET 요청 응답

6.4 WS-02 PUT 파일 업로드

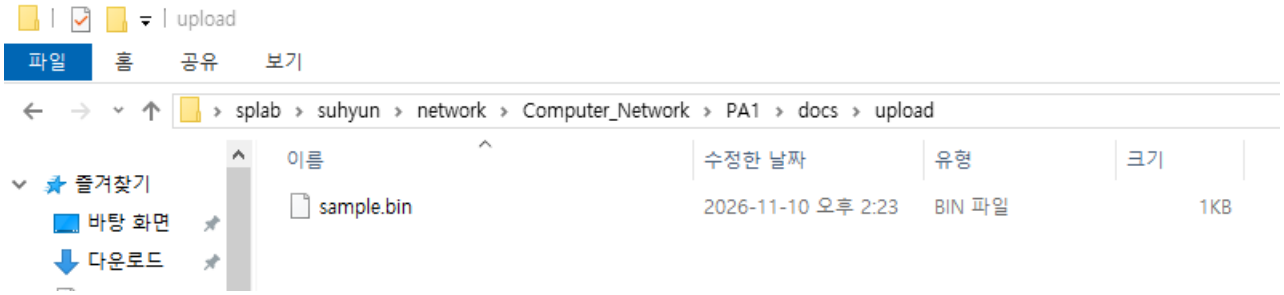
PUT http://localhost:8080/upload/sample.bin 의 Body 를 binary 로 설정하고 파일을 선택한다. 새 파일은 201 Created, 같은 이름의 덮어쓰기는 200 OK 인지 확인하고 docs/upload/sample.bin 의 내용을 원본과 비교한다.



6.3.1 최초 파일 업로드 응답



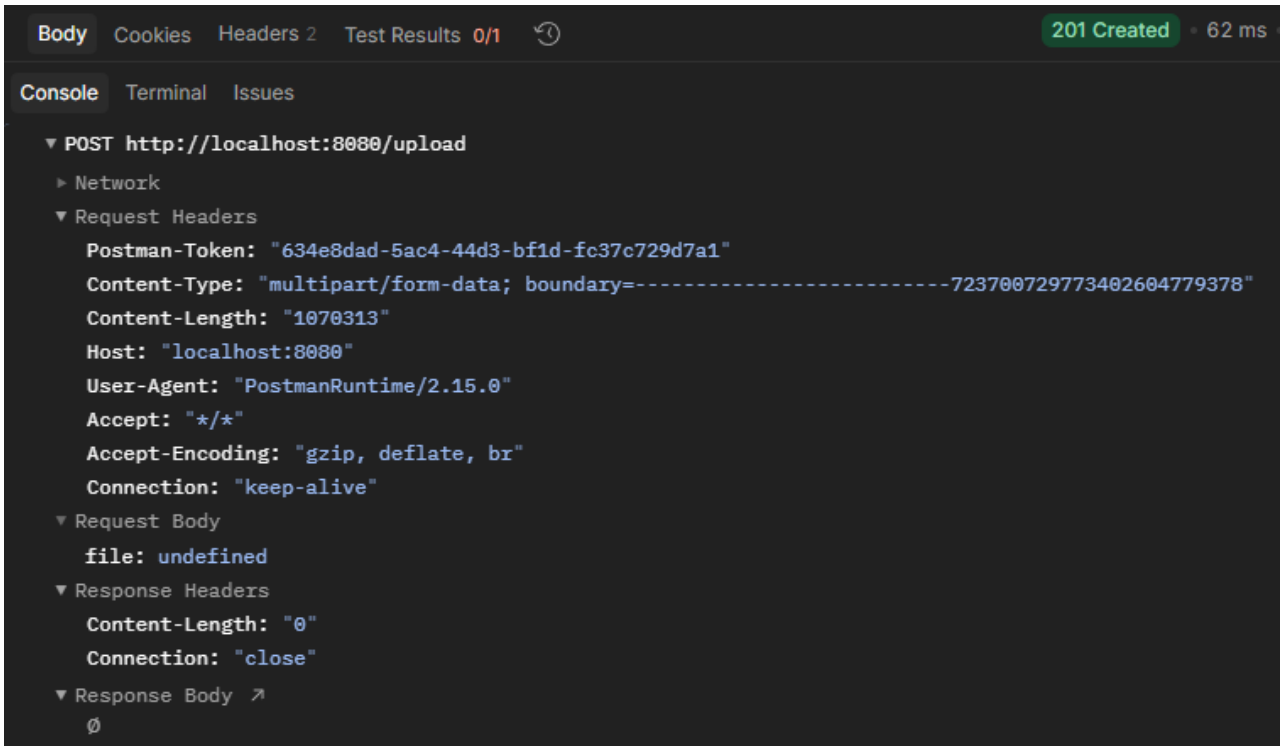
6.3.2 동일한 파일 이름 덮어쓰기



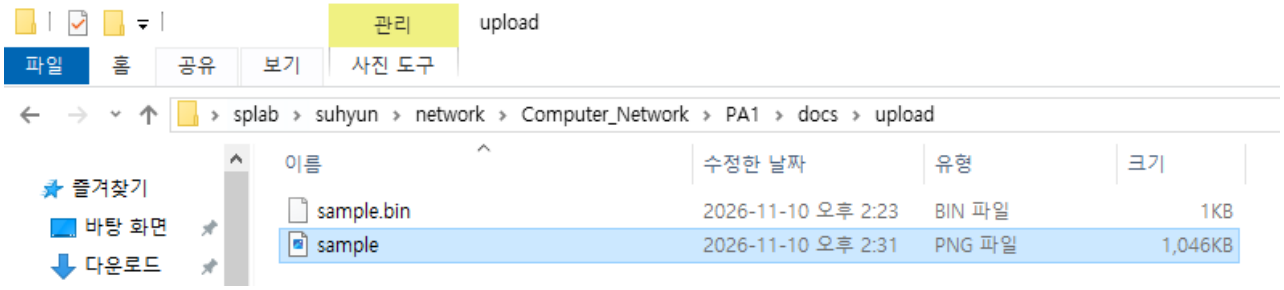
6.3.3 저장된 파일

6.5 WS-03 POST multipart 파일 업로드

POST http://localhost:8080/upload 의 Body 를 form-data 로 설정한다. key=file, type=File, value=sample.png 로 전송하여 201 Created 와 docs/upload/sample.png 저장을 확인한다. Postman Console 에서 Content-Type 의 boundary 와 file 파트 정보를 확인한다.



6.4.1 Sample.png 최초 파일 업로드



6.4.2 저장된 파일

6.6 WS-04 동시 업로드 동기화

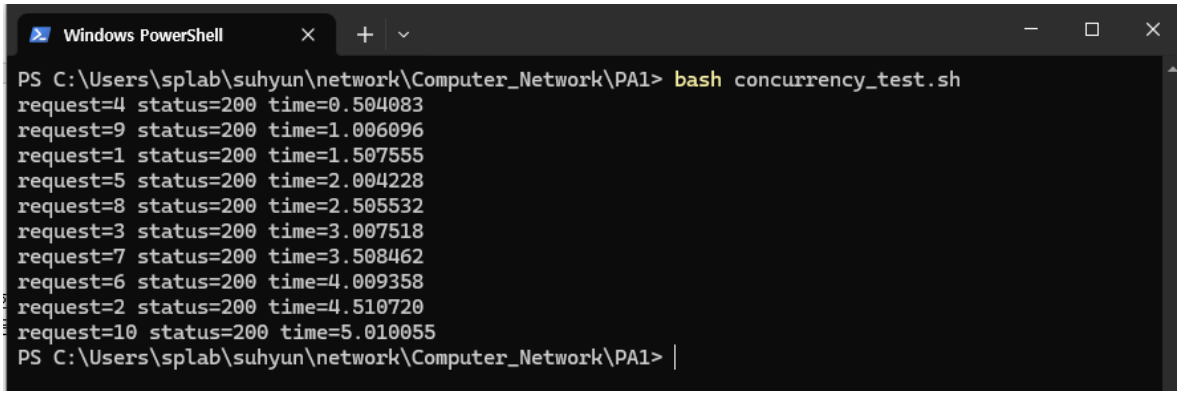
별도 테스트 폴더에서 docs/upload/shared.bin 이 없는 상태로 시작한다. 서버를 --slowDown 500 으로 실행하고, 다른 터미널에서 과제에서 제공한 concurrency_test.sh 와 sample.bin 을 같은 작업 폴더에 두어 스크립트를 실행한다.

서버:

```
java -cp out MyWebServer --port 8080 --slowDown 500
```

클라이언트:

```
bash concurrency_test.sh
```



6.5.1 클라이언트 측 로그



6.5.2 서버 측 로그